

🔗CALENDRIA - Justifications de Modélisation

Projet : CALENDRIA – Assistant de Réservation Intelligent Multi-Canal
Auteur : BOUGHERARA Safi
Formation : CDA – Concepteur Développeur d'Applications
Date : Mars 2026
Jalon : 3 – Modélisation de la Base de Données

🔗Table des Matières

- 1. [Respect de la 3ème Forme Normale \(3NF\)](#)
- 2. [Alignement avec le CDCF \(Jalon 1\)](#)
- 3. [Alignement avec la Conception UX/UI \(Jalon 2\)](#)
- 4. [Choix de modélisation détaillés](#)
- 5. [Avantages de PostgreSQL pour ce modèle](#)
- 6. [Synthèse des décisions](#)

🔗1. Respect de la 3NF

La **3ème Forme Normale** exige que :

- ✅ **1NF** : Chaque attribut est atomique (pas de groupe répétitif)
- ✅ **2NF** : Chaque attribut non-clé dépend de la totalité de la clé primaire
- ✅ **3NF** : Aucun attribut non-clé ne dépend d'un autre attribut non-clé (pas de dépendance transitive)

🔗Vérification entité par entité

Entité	1NF	2NF	3NF	Commentaire
RESTAURANT	✅	✅	✅	Tous les attributs dépendent uniquement de <code>id</code> . <code>capacite_totale</code> pourrait être calculé à partir des tables, mais il sert de plafond configurable indépendant (le restaurateur peut limiter en dessous de la capacité physique).
USER	✅	✅	✅	PK simple. <code>restaurant_id</code> est une FK, pas une dépendance transitive.
TABLE_RESTAURANT	✅	✅	✅	Tous les attributs décrivent la table elle-même. <code>restaurant_id</code> est la FK de rattachement.
SERVICE	✅	✅	✅	<code>jours_ouverture</code> est un tableau JSON, mais reste atomique du point de vue entité : c'est une propriété indivisible du service.

FERMETURE_EXCEPTIONNELLE	✓	✓	✓	Entité simple, 3 attributs métier.
CLIENT	✓	✓	✓	preferences est JSONB – atomique au sens entité (un seul bloc de préférences par client). taux_noshow est un attribut dénormalisé par choix de performance (voir ci-dessous).
RESERVATION	✓	✓	✓	3 FK distinctes. demandes_speciales dépend uniquement de cette réservation, pas du client.
CONVERSATION_IA	✓	✓	✓	telephone_appelant est conservé même si client_id existe pour couvrir le cas des appelants non encore identifiés. Ce n'est pas une redondance : le client peut ne pas être reconnu.
NOTIFICATION_SMS	✓	✓	✓	telephone_destinataire est conservé séparément de la réservation car le numéro pourrait différer du client (changement de numéro).

🔗**Cas particulier :** `taux_noshow` **(dénormalisation contrôlée)**

Le `taux_noshow` dans CLIENT est techniquement **calculable** à partir de l'historique des réservations. C'est une **dénormalisation volontaire** pour :

1. **Performance** : Éviter un `COUNT` + `GROUP BY` à chaque affichage de fiche client (utilisé dans le wireframe Fiche Client, Jalon 2)
2. **Lisibilité algorithmique** : L'algorithme d'attribution des tables (CDCF §3.1.2.C) consulte ce taux pour prioriser les clients fiables
3. **Mise à jour** : Recalculé automatiquement à chaque changement de statut de réservation (événement `noshow` ou `terminee`)

Cette dénormalisation est **acceptée en 3NF** car elle est documentée, maîtrisée, et justifiée par un besoin de performance réel.

🔗**2. Alignement avec le CDCF (Jalon 1)**

🔗**Traçabilité entités CDCF → Modèle final**

Entité CDCF (§7.2)	Entité Modèle	Statut	Justification
RESTAURANT	RESTAURANT	✓ Identique	Enrichi avec <code>duree_repas</code> et <code>buffer_nettoyage</code> (besoins de l'algorithme, CDCF §3.1.2.C)
TABLE	TABLE_RESTAURANT	✓ Renommé	<code>TABLE</code> est un mot réservé SQL – suffixe ajouté (déjà appliqué dans le code Doctrine existant)

SERVICE	SERVICE	✓ Enrichi	Fusion avec CreneauHoraire – intègre jours_ouverture
CLIENT	CLIENT	✓ Identique	Ajout de date_consentement (conformité RGPD, CDCF §4.3)
RESERVATION	RESERVATION	✓ Enrichi	Ajout de updated_at pour traçabilité des modifications
CRENEAU_HORAIRE	(fusionné dans SERVICE)	↻ Fusionné	Voir justification §4.2
CONVERSATION_IA	CONVERSATION_IA	✓ Enrichi	Ajout du champ canal pour distinguer WhatsApp/Widget/SMS (cf. 3 canaux CDCF §3.1.1)
NOTIFICATION_SMS	NOTIFICATION_SMS	✓ Identique	Conforme à la spécification CDCF §3.1.4
(non prévu)	USER	+ Ajouté	Nécessaire pour l'authentification du dashboard (CDCF §3.1.3.A)
(implicite)	FERMETURE_EXCEPTIONNELLE	+ Ajouté	Issu du wireframe Paramètres (Jalon 2) et du CDCF §3.1.2.D

🔗 Couverture fonctionnelle

Fonctionnalité CDCF	Entités impliquées	Couvert ?
§3.1.1 Module IA multi-canal	CONVERSATION_IA, CLIENT	✓
§3.1.2 Algorithme tables	TABLE_RESTAURANT, SERVICE, RESERVATION	✓
§3.1.3 Dashboard restaurateur	USER, RESTAURANT, RESERVATION	✓
§3.1.3.D Gestion des réservations	RESERVATION, CLIENT, TABLE_RESTAURANT	✓
§3.1.3.F Horaires & Services	SERVICE, FERMETURE_EXCEPTIONNELLE	✓
§3.1.3.G Gestion des clients	CLIENT (preferences, taux_noshow, RGPD)	✓
§3.1.4 Notifications SMS	NOTIFICATION_SMS	✓
§3.1.5 Base de données clients	CLIENT (RGPD, historique)	✓
§4.3 Sécurité OWASP	USER (password hashé, roles)	✓
§4.3 RGPD	CLIENT (consentement_rgpd, date_consentement)	✓

🔗3. Alignement avec la Conception UX/UI (Jalon 2)

🔗Correspondance wireframes → entités

Écran (Jalon 2)	Données affichées	Entités source
Login	email, password	USER
Dashboard	Nb réservations, couverts, taux remplissage, prochaines arrivées	RESERVATION, TABLE_RESTAURANT, RESTAURANT
Calendrier (jour/semaine/mois)	Réservations par date, code couleur statut	RESERVATION, CLIENT
Plan de Salle	Tables avec statut temps réel, timeline Gantt	TABLE_RESTAURANT, RESERVATION
Réservations (liste)	Heure, client, personnes, table, statut, filtres	RESERVATION, CLIENT, TABLE_RESTAURANT
Fiche Client	Nom, tel, email, préférences, taux no-show, historique	CLIENT, RESERVATION
Statistiques	Taux remplissage, analyse no-show, export	RESERVATION, CLIENT, TABLE_RESTAURANT
Paramètres > Horaires	Services midi/soir, jours ouverture, fermetures	SERVICE, FERMETURE_EXCEPTIONNELLE
Widget Chatbot	Conversation IA, sélecteurs	CONVERSATION_IA, CLIENT, RESERVATION

Conclusion : Chaque donnée visible dans les wireframes est stockée dans au moins une entité du modèle. Il n'y a pas de donnée « orpheline ».

🔗4. Choix de Modélisation

🔗4.1 Séparation User / Client

Choix : Deux entités distinctes sans lien direct.

Pourquoi ?

- **User** = compte authentifié (email + password hashé + rôles Symfony Security)
- **Client** = personne qui réserve (identifié par téléphone, pas de mot de passe)

Avantages :

1. Respect du **Single Responsibility Principle** : cycle de vie et attributs complètement différents
2. **Sécurité** : `User` implémente `UserInterface` de Symfony, `Client` n'a aucune notion d'authentification
3. **RGPD** : Le droit à l'oubli du Client ne doit pas impacter les comptes User du dashboard
4. **Scalabilité** : En multi-restaurants (V2), un User pourrait gérer plusieurs restaurants, indépendamment des clients

🔗 4.2 Fusion CreneauHoraire dans Service

Choix : Le CDCF prévoyait `CRENEAU_HORAIRE` (jour_semaine, heure_debut, heure_fin, actif). Cette entité est **fusionnée dans** `SERVICE` via le champ `jours_ouverture` (JSON array).

Pourquoi ?

- Le code Symfony existant utilise déjà cette structure (`Service` avec `joursOuverture: array`)
- Séparer les deux créerait un mapping 1:N entre `Service` et `CreneauHoraire` où chaque `CreneauHoraire` ne serait qu'un jour de la semaine → **sur-normalisation**
- Avec 2 services (midi/soir) × 7 jours, cela ferait 14 lignes au lieu de 2, sans gain fonctionnel

Avantages :

1. Modèle plus simple et lisible
2. Cohérence avec le code existant
3. JSONB PostgreSQL permet des requêtes efficaces sur les jours (`@>` operator)
4. Le wireframe "Paramètres > Horaires" (Jalon 2) affiche les jours comme des checkboxes liées au service, pas comme des entités séparées

🔗 4.3 Ajout de FermetureExceptionnelle

Choix : Entité séparée plutôt qu'un champ JSON dans Restaurant.

Pourquoi ?

- Le wireframe "Paramètres" (Jalon 2) montre une **liste dynamique** de fermetures avec ajout/suppression
- L'algorithme de disponibilité (CDCF §3.1.2.D) doit pouvoir **requêter efficacement** les fermetures par date
- Un champ JSON dans Restaurant violerait la 1NF (groupe répétitif contenant des dates)

Avantages :

1. Respect de la 3NF
2. Requêtes SQL simples : `WHERE date_fermeture = '2026-12-25'`
3. Contrainte UNIQUE (`restaurant_id, date_fermeture`) → pas de doublon
4. Index sur `date_fermeture` pour des vérifications rapides

🔗 4.4 JSONB pour les préférences client

Choix : `preferences JSONB` plutôt qu'une table séparée `CLIENT_PREFERENCE` .

Pourquoi ?

- Les préférences sont **semi-structurées** : allergies (liste variable), demandes spéciales, notes
- Créer une table `CLIENT_PREFERENCE` avec des lignes (`client_id, type, valeur`) serait un **modèle EAV** (Entity-Attribute-Value) qui est reconnu comme un anti-pattern en modélisation relationnelle
- Les préférences sont toujours lues/écrites **en bloc** avec la fiche client

Avantages :

1. Flexibilité : ajout de nouvelles préférences sans modification du schéma
2. Performance : une seule lecture pour toute la fiche client

- 3. Index GIN : permet la recherche « tous les clients allergiques au gluten » via `preferences @> '{"allergies": ["gluten"]}'`
- 4. Cohérence avec le code Doctrine existant (`private ?array $preferences`)

4.5 table_id nullable dans Reservation

Choix : `table_id` est **nullable** dans RESERVATION.

Pourquoi ?

- Le CDCF (§7.2) précise : `id_table` (FK → TABLE, NULLABLE initialement)
- La réservation est d'abord **créée** par le chatbot, puis la table est **attribuée** par l'algorithme
- En cas de forte demande, plusieurs réservations peuvent être en attente d'attribution

Avantages :

1. Découplage entre création et attribution (workflow en deux temps)
2. L'algorithme d'attribution peut tourner de manière asynchrone
3. `ON DELETE SET NULL` : si une table est supprimée, les réservations ne sont pas perdues

5. Avantages de PostgreSQL

Le CDCF (§4.2) recommandait PostgreSQL. Voici comment le modèle en tire parti :

Feature PostgreSQL	Usage dans CALENDRIA	Avantage
JSONB	<code>preferences</code> , <code>roles</code> , <code>jours_ouverture</code> , <code>donnees_collectees</code>	Données semi-structurées indexables, flexibles
GIN Index	Index sur <code>preferences</code> (client)	Recherche dans les objets JSON sans scan complet
TIMESTAMPTZ	Tous les <code>created_at</code> , <code>date_debut</code> , <code>date_envoi</code>	Gestion correcte des fuseaux horaires
CHECK constraints	Validation de <code>statut</code> , <code>type</code> , <code>nombre_personnes > 0</code>	Intégrité des données au niveau SGBD
Partial UNIQUE	<code>id_externe_twilio</code> unique seulement si non NULL	Unicité conditionnelle élégante
SERIAL	Toutes les PK	Auto-incrémentation native
ON DELETE CASCADE/SET NULL	Relations parent-enfant	Gestion précise de la suppression en cascade
Array et opérateurs	<code>@></code> sur JSONB	Requêtes riches sur les données semi-structurées

6. Synthèse des Décisions

#	Décision	Justification	Bénéfice
1	9 entités (vs 8 CDCF)	+User (auth), +Fermeture (wireframes), -CreneauHoraire (fusionné)	Couverture complète sans sur-normalisation
2	User ≠ Client	Rôles métier et cycles de vie distincts	SRP, sécurité, RGPD
3	CreneauHoraire → Service	JSON <code>jours_ouverture</code> dans Service	Simplification, cohérence code
4	FermetureExceptionnelle séparée	Requêtes par date, wireframe dynamique	1NF respectée, index performant
5	JSONB pour preferences	Semi-structuré, variable, bloqué par client	Flexibilité, GIN index
6	table_id nullable	Attribution différée par l'algorithme	Workflow chatbot en 2 temps
7	taux_noshow dénormalisé	Performance lecture, algorithme attribution	Accès O(1) vs requête agrégée
8	PostgreSQL	JSONB, GIN, TIMESTAMPTZ, CHECK, Partial UNIQUE	Modèle riche et performant
9	Nommage <code>TABLE_RESTAURANT</code>	Mot réservé SQL	Compatibilité SQL standard
10	ON DELETE CASCADE/SET NULL	Cascade pour config, SET NULL pour données métier	Intégrité référentielle adaptée